FPGA Architectures to Enable Concurrent LUT and Adder Chain Usage

XILAI DAI, Cornell University, New York, New York, USA
JUNIUS PUN, Nanyang Technological University, Singapore, Singapore
ANTHONY FEI and JULIAN BUSHLOW, Cornell University, Ithaca, New York, USA
GRACE ZGHEIB and MAHESH A. IYER, Altera, San Jose, California, USA
MOHAMED ELDAFRAWY, Groq Inc., Mountain View, California, USA
ANDREW BOUTROS, University of Waterloo, Waterloo, Ontario, Canada
VAUGHN BETZ, University of Toronto, Toronto, Ontario, Canada
MOHAMED S. ABDELFATTAH, Cornell University, New York, New York, USA

Flexibility and customization make Field-Programmable Gate Arrays (FPGAs) attractive for arbitrary-precision arithmetic-heavy workloads such as sparse and mixed-precision DNN inference. However, in modern logic blocks, the hardware adder chain is typically driven only by look-up table (LUT) outputs, preventing adders and LUTs from being used independently and concurrently within a logic element, and reducing the hardware resource utilization efficiency. We propose *Double-Duty*, a logic block architecture that decouples LUTs and adder chains by augmenting four extra inputs of the logic element to bypass LUTs and directly drive the adders. We model the added circuits at the transistor level and implement full CAD support in open source tools, including improved arithmetic synthesis to provide a realistic baseline. On a Stratix-10-like architecture, Double-Duty reduces area by 21.6% on Kratos adder-intensive circuits and by 9.3% and 8.2% on the Koios and VTR suites, respectively, improving average area-delay product by 9.7% across all benchmarks with minimal impact on critical path delay. Additionally, we combined Double-Duty with prior arithmetic-oriented optimized architectures, and the fused designs achieve up to 20% area-delay product reduction in geometric mean across multiple benchmarks compared to the baseline.

CCS Concepts: • **Hardware → Integrated circuits**; **Reconfigurable logic and FPGAs**; **Programmable logic elements**;

Xilai Dai, Junius Pun, and Anthony Fei contributed equally to this work.

This project is supported by Intel Corporation, the National Science Foundation under Grant No. 2303626, and the CN Yang Scholars Programme, Nanyang Technological University.

Authors' Contact Information: Xilai Dai (corresponding author), Cornell University, New York, New York, USA; e-mail: xd44@cornell.edu; Junius Pun, Nanyang Technological University, Singapore, Singapore; e-mail: JPUN001@e.ntu.edu.sg; Anthony Fei, Cornell University, Ithaca, New York, USA; e-mail: ayf7@cornell.edu; Julian Bushlow, Cornell University, Ithaca, New York, USA; e-mail: jb2698@cornell.edu; Grace Zgheib, Altera, San Jose, California, USA; e-mail: grace.zgheib@altera.com; Mahesh A. Iyer , Altera, San Jose, California, USA; e-mail: mahesh.iyer@altera.com; Mohamed Eldafrawy, Nvidia, Santa Clara, California, USA; e-mail: meldafrawy@nvidia.com; Andrew Boutros, University of Waterloo, Waterloo, Ontario, Canada; e-mail: andrew.boutros@uwaterloo.ca; Vaughn Betz, University of Toronto, Toronto, Ontario, Canada; e-mail: vaughn@ece.utoronto.ca; Mohamed S. Abdelfattah, Cornell University, New York, New York, USA; e-mail: mohamed@cornell.edu.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).
ACM 1936-7414/2026/12-ART50
https://doi.org/10.1145/3834569

Additional Key Words and Phrases: FPGA architecture, logic block, adder chain, LUT bypass, VTR CAD

ACM Reference format:
Xilai Dai, Junius Pun, Anthony Fei, Julian Bushlow, Grace Zgheib, Mahesh A. Iyer, Mohamed Eldafrawy, Andrew Boutros, Vaughn Betz, and Mohamed S. Abdelfattah. 2026. FPGA Architectures to Enable Concurrent LUT and Adder Chain Usage. *ACM Trans. Reconfig. Technol. Syst.* 19, 4, Article 50 (December 2026), 25 pages. https://doi.org/10.1145/3834569

1 Introduction

Field-Programmable Gate Arrays (FPGAs) offer bit-level programmability at the price of performance when compared to Application-Specific Integrated Circuits. This enables their flexible use in many application domains where deployment-time reconfigurability is warranted, such as cloud networking [20, 38, 43], wireless communication [23, 24, 32], and machine learning [1, 41, 42, 46]. Many mainstream machine learning workloads depend on high-performance dense matrix multiplications, pushing FPGA vendors to either bolster their FPGAs with tensor compute units [7, 31] or resort to heterogeneous FPGA-CGRA hybrid computing devices [21]. However, as machine learning becomes ever more prevalent, it is finding its way into a broad variety of applications that can take advantage of the flexibility of the FPGAs' soft logic. This is highlighted by the plethora of works to create new frameworks [17], methods [4, 39–41], benchmarks [16], and architectures [9, 36] to improve the efficiency of **Deep Neural Networks (DNNs)** implemented using the FPGA fabric soft logic.

Soft logic remains important even on FPGAs with DSPs and other hard blocks, because hard blocks are optimized for a finite set of modes and bit widths. For example, DSP units commonly target fused multiply-add operations with operand widths such as 18, 27, or 32 bits [3, 26]. Prior work has made FPGA hard blocks more reconfigurable [5, 10, 13, 14], but soft logic still provides finer-grained bit-level specialization. This is particularly useful for inference workloads that use sparsity, low precision, or custom numerical formats, where synthesis can remove ineffectual operations at the operand and bit levels [16, 41] or implement specialized arithmetic formats [12, 30]. For instance, when synthesizing a multiplication operation where one operand is known at compile time, the operation can often be decomposed into a series of additions, allowing better utilization of FPGA adder chains. Furthermore, adder trees are important and common as they are needed to perform reduction in matrix multiplication.

Several prior works have explored arithmetic efficiency in FPGA soft logic. Compressor trees, constructed with **Generalized Parallel Counters (GPCs)** [25, 29, 34], combine the use of both **Look-up Tables (LUTs)** and dedicated adders to further boost resource utilization. Other approaches involve architectural changes to the **Logic Block (LB)**, such as packing more adders in one logic element [18] or including explicit XOR gates to improve compressor tree implementation [36]. However, current FPGA architectures present a fundamental limitation: the adder carry chain can only be driven by LUT outputs. Consequently, LUTs and adder chains must implement closely related logic to effectively utilize both resources. If a circuit uses many adder chains—as is often the case with matrix multiply reduction operations—the associated LUTs may be occupied only to feed adder inputs rather than to implement independent logic. Conversely, if LUTs implement unrelated logic functions, the adder chain remains inaccessible within that logic element even when the surrounding LB still has routable inputs and outputs.

To address this shortcoming, we propose a modification to FPGA LBs to more flexibly enable the concurrent use of adder chains and LUTs. We fully implement our architecture—Double-Duty—in open source FPGA CAD tools, from circuit and architectural modeling, to synthesis algorithms, to

enable an extensive evaluation over three popular FPGA benchmark suites. Our results consistently show that the extra flexibility of enabling concurrent and independent use of LUTs and adder chains makes Double-Duty significantly more area-efficient compared to current FPGAs without compromising critical path delay. More concretely, we make the following contributions[1]:

(1) We propose the Double-Duty FPGA LB architecture, which decouples the connections between LUTs and adders, enabling their independent and concurrent usage.
(2) We quantify the area and critical path delay of the additional circuit components introduced by the Double-Duty architecture using SPICE-based simulation and transistor sizing with **Circuit Optimization for FPGA Exploration (COFFE)** [15], and we perform an ablation study on reducing this overhead.
(3) We integrate compressor tree algorithms into the open source FPGA CAD tool **Verilog-to-Routing (VTR)** [19], significantly improving efficiency when synthesizing adder chains, ensuring a strong baseline on which to evaluate Double-Duty.
(4) We evaluate our Double-Duty architecture on a suite of comprehensive benchmarks, including the VTR standard benchmarks, Koios ML benchmarks [6], and Kratos unrolled DNN benchmarks [16], achieving an average 9.7% improvement in **Area-Delay Product (ADP)** over all circuits, and up to an 80% increase in packing density in stress tests.
(5) We implement multiple architecture variants proposed in prior work to enhance arithmetic support in FPGAs, and we add support for these architectures within the VTR flow to be able to evaluate large-scale benchmarks. We compare and combine these new architectures with Double-Duty and explore the efficiency tradeoffs on Kratos, Koios, and the VTR benchmark sets.

2 Background

2.1 FPGA Basics

FPGAs can implement any logic function through programmable LUTs, organized into large clusters called LBs. Every LB contains multiple (typically 10) smaller units, called **Adaptive Logic Modules (ALMs)** or Fracturable Logic Elements, connected to a programmable interconnect network via a local crossbar. Figure 2(a) shows the ALM design typical of Intel's Stratix 10 series FPGAs. Each ALM typically consists of four 4-input LUTs that can be combined with multiplexers to implement two 5-input truth tables or a single 6-input truth table. In addition to LUTs, each ALM contains two 1-bit full adders, whose inputs are directly connected to the output of 4-LUTs. This structure helps simplify logic before addition, and the LUT can be used in many addition algorithms [25, 34]. Moreover, the carry-in and carry-out signals are connected along multiple ALMs, forming a long carry chain, allowing fast and high-bit-width integer addition.

Despite this versatility, a key limitation of this conventional FPGA architecture is that the LUTs and adders are not independent. When the adders are in use, the LUT outputs provide inputs to the adder chain and thus cannot be used to implement other logic functions. This dependence limits the resource utilization in many arithmetic-heavy workloads. Some of the recent commercial FPGAs from Xilinx [2] have completely removed the 1-bit adders and instead use LUTs to generate

[1] An earlier version of this work appeared at the 2025 International Conference on Field-Programmable Logic and Applications [35]. In this expanded journal version, we added three main investigations: (1) An ablation on the input sharing between our fast adder chain inputs and the existing LUT inputs. (2) New VTR architecture descriptions, packing support, and synthesis-flow support for multiple architecture variants from prior work [9], enabling complete benchmark runs within VTR [19] and direct comparison to our proposed Double-Duty architecture. (3) Combination of Double-Duty with ideas from prior architecture variants to explore efficiency tradeoffs on large benchmark sets.

carry-propagate and generate signals, which further limits the ability of using arithmetic and logic resources independently.

2.2 CAD Tools

VTR. VTR [19, 33] is an open source FPGA CAD tool that takes Verilog design files with an FPGA architecture description file and performs synthesis, placement, routing, and timing analysis. By using a tree-structured XML architecture description file, VTR allows users to experiment with arbitrary FPGA architectures and quantitatively evaluate new architectures and CAD algorithms.

Parmys. VTR has undergone several enhancements over time. Recently, it changed the synthesis front-end from Odin II [27] to Yosys [44], a more flexible and advanced RTL synthesis tool that supports modern features like SystemVerilog generate statements. However, as the Yosys front-end is a general-purpose synthesis tool, VTR relies on a plugin called Parmys (Partial Mapper for Yosys) to handle FPGA-specific technology mapping [19]. Parmys has some significant limitations, particularly in the optimization of arithmetic operations including addition and multiplication. In this article, we develop adder chain synthesis within Parmys to significantly improve its efficiency in synthesizing adder chains and compressor trees, to be able to investigate our proposed Double-Duty architecture using a strong CAD baseline.

COFFE 2. COFFE [15] is an automated transistor-level modeling tool that provides accurate estimates of area, delay, and energy consumption for FPGA tiles by utilizing HSPICE simulations and automated transistor sizing. Its successor, COFFE 2 [45], extends these capabilities to heterogeneous FPGA architectures, supporting complex LBs, fracturable LUTs, and custom DSP tiles. In this work, we use COFFE 2 to model our Double-Duty architecture for precise area and timing estimation.

2.3 Arithmetic Optimizations

Integer multiplications are the core of many computation workloads, especially in machine learning inference. While DSPs are specialized units dedicated to multiplication, they are a limited resource and are usually better utilized with high bit width and when both operands are unknown at compile time. However, a soft logic multiplier is still crucial to optimize resource utilization, particularly for custom and low-bit width arithmetic, and especially for unrolled DNN inference circuits [16, 39, 40] in which one operand is known at compile time from the trained model parameters. This reduces a multiplication operation into multiple additions of partial products. For an n-bit by n-bit multiplication, n such rows are formed, which require $n - 1$ summations. Using only full adders, an approach to reduce latency is to use binary adder trees, which perform as many summations as possible in parallel, requiring $O(\log n)$ time to obtain the final result. This is currently the default and only approach taken by VTR to synthesize soft multipliers. To exploit the available LUTs in FPGAs, compressor trees [25, 29, 34] can also be used for efficient summation. Carry-save logic, implemented with LUTs, is used to compress the initial n rows into two final rows that can be summed together with a fast ripple-carry chain. Figure 1 illustrates the conceptual diagrams of two widely used algorithms, Wallace and Dadda, with Cascade—a simple algorithm that only uses adder chains—for comparison. Notably, compressor trees are currently not supported in Parmys/VTR. This absence is not just an implementation detail: VTR's modern front end uses Yosys and Parmys to map arithmetic into FPGA primitives, and Parmys historically lowered multipliers through binary adder structures rather than architecture-aware compressor tree reductions. As a result, open source FPGA architecture studies that depend on soft multipliers can overestimate the LUT and adder resources required or can miss architectures whose benefit appears only when LUT-based compression and carry chain addition are used together. Adding compressor tree synthesis in this work enables VTR to evaluate soft-logic implementations of constant multipliers, multiplier reductions, and DNN inference data paths with a stronger open source arithmetic baseline.

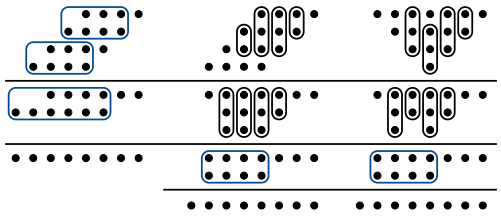


Fig. 1. A conceptual diagram of a 4-bit multiplier using Cascade, Wallace, and Dadda algorithms. Pill shapes represent LUT-based compressors, and rectangles represent full adder chains. Each layer represents a reduction stage.

3 Double-Duty Architecture and Circuit-Level Modeling

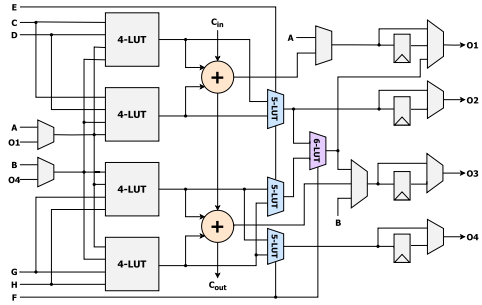
This section presents our new architecture, Double-Duty, that enables the concurrent use of LB LUTs and adder chains, resulting in denser FPGA placement. We describe the architectural enhancements and variants, evaluate our proposal at the circuit level, and present the CAD synthesis changes that were needed to effectively and fairly assess our proposed architecture.

3.1 Double-Duty LB Architecture

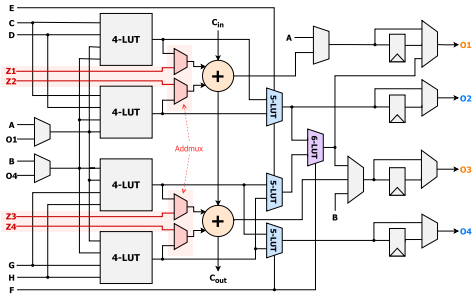
To enable independent and concurrent use of both LUTs and adders, we introduce two main changes to the ALMs and local routing as described below. We focus on *DD5*, which enables 5-LUT mode concurrently with the adders, and also evaluate *DD6* as an alternative configuration to test whether supporting concurrent 6-LUT mode provides additional practical benefit.

AddMux. In each ALM, we add extra multiplexers and four additional ALM inputs (Z1–Z4), as highlighted in Figure 2(b). These modifications allow inputs to bypass the LUTs and connect directly to the adder chain, enabling the use of adders and LUTs concurrently and independently. In this revised architecture, the ALMs support independent operation in 5-LUT mode: two output pins are allocated for adder outputs (O1 and O3), while the remaining two output pins can be used for the 5-LUT outputs (O2 and O4). The availability of separate 5-LUT output pins is inherited from the Stratix-10-style ALM; the new capability is that the adders no longer consume the LUT outputs as their required inputs, so these pre-existing output pins can now carry unrelated 5-LUT logic while the adders are active.

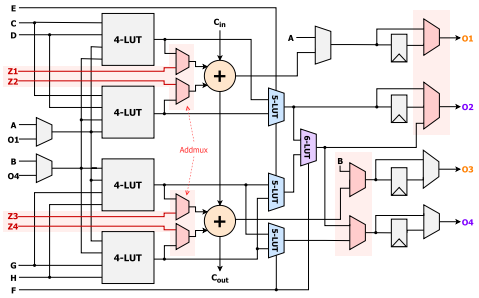
AddMux Crossbar. To accommodate the additional ALM inputs (Z1–Z4), we introduce a secondary local interconnect, which is sparsely populated and draws signals from the existing LB input pool as shown in Figure 3. Importantly, the total number of input pins to the LB remains unchanged—this modification leverages the same inputs of the original LB and therefore does not require any modifications to global routing. Notably, the wires that our new crossbar connects to have connections from LB-to-LB direct links [28]. Specifically, Stratix-10 architectures include 40 LB-to-LB wires, 20 from the east LB and 20 from the west LB. These enable direct connections between adjacent LBs without needing to access the global interconnect. As shown in Figure 3, these direct LB-to-LB connections are multiplexed together with global routing into the connection block multiplexers. These feed 60 LB inputs, 40 of which can be connected to LB-to-LB inputs. We connect the AddMux Crossbar to 10 of the 60 LB input wires, chosen from the subset that can also receive LB-to-LB direct links. Equivalently, each AddMux input sees only $\frac{10}{60}$ of the LB input pool, so this crossbar is 17% populated. This sparse population is a deliberate area tradeoff: it gives the new Z inputs access to nearby and global signals without replicating the much larger local crossbar, whose population exceeds 50% in the reference Stratix-10-like architecture [28].



(a) Baseline Stratix-10 ALM.



(b) Double-Duty ALM. (DD5)



(c) Double-Duty ALM with concurrent LUT6 mode. (DD6)

Fig. 2. Baseline (a) and Double-Duty ALM architectures. We show two variants: the DD5 (b) supports concurrent 5-LUT and adder usage while the DD6 (c) supports concurrent 6-LUT and adder usage.

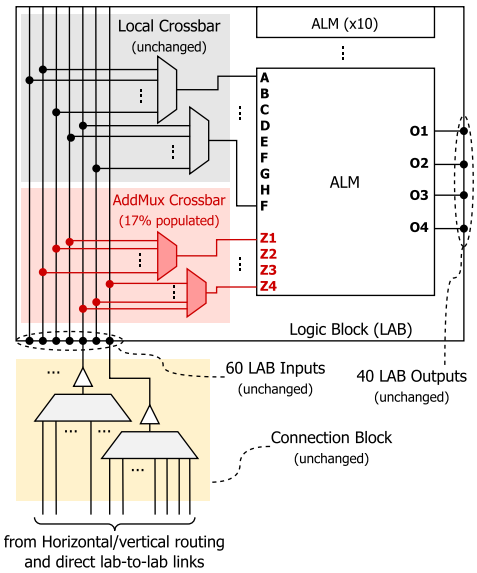


Fig. 3. A new AddMux Crossbar connects existing LB inputs to the new adder chain direct paths (Z1–Z4).

Table 1. Area and Delay of Added Circuit Components

Circuit	Area (MWTAs)	Delay (ps)
AddMux	1.698	68.77
Baseline Crossbar	289.6	72.61
AddMux Crossbar	77.91	77.05
Baseline ALM	2,167.3	-
DD5 ALM	2,248.0 (+3.72%)	-

Area is reported per ALM in MWTAs. MWTAs, minimum-width transistor areas.

DD6. The AddMux and AddMux Crossbar allow the concurrent usage of 5-LUTs and adders; however, DD5 does not support concurrent 6-LUT usage with the adders. To address this, we propose an extended version, DD6, which modifies the multiplexing of LUT and adder signals to the outputs, enabling fully independent 6-LUT and adder operation, as shown in Figure 2(c). However, our preliminary analysis indicates that the additional flexibility provided by DD6 yields marginal benefits in practice. A detailed evaluation of these tradeoffs is presented in Section 5.

3.2 Circuit-Level Modeling of Double-Duty

To model the delay and area overheads introduced by our architectural modifications, we build upon the framework established by Eldafrawy et al. [18], leveraging COFFE 2 [45] to replicate the Stratix-10-like baseline architecture used in their work. We first express the new ALM paths and local-interconnect connections in the VTR XML architecture model, then mirror the corresponding mux structures in COFFE 2 so that each added XML-level path has a transistor-level circuit model. In particular, we create SPICE models for AddMux, as well as the additional AddMux Crossbar. COFFE 2 is then used to automatically size the transistors, and the resultant area and delay characteristics are used in the VTR model for the benchmark-level evaluations.

Table 1 shows the area and delay costs of each added component in the Double-Duty ALM shown in Figure 2. Overall, our modifications increase the tile area by 3.72% as compared to the baseline architecture. Table 2 shows the delay impact of the added components on several paths within the architecture. Although these multiplexers cause a LUT-to-adder delay increase, the delay of any signal feeding the adders directly is almost cut in half, as now the signal does not need to go through the LUT anymore. This 51.6% increase on the conventional A-H → adder path is the main timing cost of preserving compatibility with LUT-fed addition. In practice, Double-Duty gives the CAD tool two choices: use the conventional LUT-fed path when logic absorption before addition is valuable, or use the direct Z-input path when the adder inputs do not need LUT pre-processing. In addition, the delay of LUT output to ALM output remains almost the same as the output circuitry is largely unchanged. Note that Table 2 shows that the AddMux Crossbar has higher delay than the existing Local Crossbar. This may seem counterintuitive, as the AddMux Crossbar is much smaller; however, this additional delay is an artifact of transistor sizing in COFFE 2. COFFE 2 tries to optimize all the paths going through an ALM, and the Z ALM inputs have a much lower delay to the output of the ALM compared to the conventional LUT inputs; therefore, under the same area-delay optimization objective, COFFE 2 spends more transistor area on the Local Crossbar and sizes the AddMux Crossbar more conservatively. The result should therefore be read as the optimized design point produced by our modeling objective, not as evidence that a small crossbar is intrinsically slower than a larger one.

Table 2. Delay Impact of Added Circuits in the Double-Duty Architecture Variants on Data Paths

Architecture	Path	Delay (ps)
Baseline	LB input → ALM inputs A-H ❶	72.61
	ALM inputs A-H → Adder input ❷	133.4
Double-Duty	LB input → ALM inputs Z_1-Z_4	77.05 (+6.11% vs. ❶)
	ALM inputs A-H → Adder input	202.2 (+51.6% vs. ❷)
	ALM inputs Z_1-Z_4 → Adder input	68.77 (−48.4% vs. ❷)

4 VTR CAD Enhancements

Soft multiplication synthesis in VTR has not been fully optimized, which results in redundant and/or wasteful resource usage. To tackle these shortcomings, we integrated several optimization techniques into VTR's synthesis. These changes make the baseline architecture more competitive than unmodified VTR for arithmetic-heavy circuits, so improvements attributed to Double-Duty are measured against a stronger CAD flow rather than against avoidable synthesis inefficiencies.

Unrolled Multiplication. General multiplication treats each partial product as a unique signal, since its value is unknown during compilation. However, in the case of unrolled multiplication, the rows are shifted duplicates of the multiplicand and are included in the summation if the corresponding bit of the known operand is "1." We define this corresponding bit as the "selector bit" of the row. When forming partial sums from these rows, there can therefore be adder chains that have identical input signals but sum different rows. Instead of synthesizing duplicate adder chains, which is the current VTR behavior, a single adder chain can be used instead, with its output signals fanned out as inputs for future adder chains. Multiplicand rows can also be excluded if their selector bits are "0," reducing the initial number of rows to sum. For a sample 8-bit multiplication with a $(01010101)_2$ constant, baseline VTR uses 2.85× more full adders than in the redundancy-aware implementation. We compute this ratio by counting the full-adder primitives emitted by baseline VTR for the constant-multiplier netlist and dividing by the count after structurally identical adder chains are shared.

Improved Binary Adder Tree Synthesis. For n rows to sum, it is possible to insert $\lfloor n/2 \rfloor$ parallel adder chains at stage 1 of the addition operation, whose k rows of outputs form the next k rows to reduce in stage 2. Adder chains need not necessarily be formed from adjacent rows, which presents many possible adder chain combinations. To find the best combination at every stage, a *strength heuristic* is defined to reduce the ratio of included signals by the adder chains to the number of output signals generated in this stage. Included signals are counted by position, meaning that a duplicate input signal in different rows can be counted multiple times. Produced output signals are counted by unique adder chain, so duplicate chains do not increase the output count. The *strength* metric is therefore higher when a placement consumes many positioned row signals but produces fewer unique chains after redundancy is removed. Figure 4 demonstrates this concept on a 6-bit example; the bit width is chosen only to make the duplicate-chain structure visible in a small figure, while the algorithm itself is not specific to 6-bit multiplication. To determine the best placement under this per-stage heuristic, we use the dynamic programming approach in Algorithm 1. This row-selection procedure is part of our VTR implementation; it is not a new multiplier-reduction theory, but a CAD heuristic for choosing which rows to combine so that redundant adder chains can be detected and shared.

Compressor Tree Synthesis. Instead of relying solely on adders, compressor trees implement reduction operations using both LUTs and adder chains. Current compressor tree implementations on FPGAs rely on GPCs specific to each architecture [25]. To implement compressor trees for any arbitrary architecture, each output signal of a compressor can be viewed as the result of a Boolean

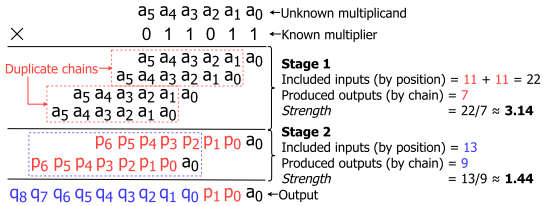


Fig. 4. A sample 6-bit by 6-bit unrolled multiplication, where one operand is a compile-time constant and only selected shifted rows of the other operand must be summed. With known multipliers, duplicate adder chains can appear in reduction stages. The *strength* of each reduction stage is computed by counting included input signals by row position and produced output signal by unique adder chain.

Algorithm 1: Adder Row Selection for Maximum Strength

$C_S \leftarrow$ empty collection ▷ empty solution cache
procedure BESTPLACEMENT(R_n) ▷ n-subset of rows
  if $n = 2$ **then return** R_n
  end if
  if $C_S(R_n)$ exists **then return** $C_S(R_n)$
  end if
  $S_{\text{best}} \leftarrow$ null ▷ best solution
  if n is even **then**
    for $p \leftarrow$ all pairs in R_n **do**
      $S_{R_{n-2}} \leftarrow$ BESTPLACEMENT($R_{n-2} \leftarrow R_n \backslash \{p\}$)
      $A_p \leftarrow$ adder chain with p as inputs
      $I_p, O_p, I_S, O_S \leftarrow$ input, output signals of A_p, $S_{R_{n-2}}$
      $I_S \leftarrow I_S + I_p$
      if A_p is not used in $S_{R_{n-2}}$ **then** $O_S \leftarrow O_S + O_p$
      end if
      if $H_S = I_S/O_S > H_{S_{\text{best}}}$ or S_{best} is null **then**
        $S_{\text{best}} \leftarrow S\{A_p, S_{R_{n-2}}\}$
      end if
    end for
  else if n is odd **then**
    for $r \leftarrow R_n$ **do** ▷ for each row
      $S_{R_{n-1}} \leftarrow$ BESTPLACEMENT($R_{n-1} \leftarrow R_n \backslash \{r\}$)
      if $H_{S_{R_{n-1}}} > H_{S_{\text{best}}}$ or S_{best} is null **then** $S_{\text{best}} \leftarrow S_{R_{n-1}}$
      end if
    end for
  end if
  Add S_{best} to C_S, **return** S_{best}
end procedure

equation of the GPC's input signals. After inserting compressors at each reduction stage, the final two rows are summed together with an adder chain. The intermediate combinational logic can then be optimized as part of logic synthesis and then packed into LUTs. ABC synthesis [11] is responsible for this logic packing within the VTR suite, and thus, only the initial Boolean logic mapping from the soft multiplier's input signals to the final adder chain's input signals needs to be implemented in VTR. To achieve this, we make use of two compressor tree algorithms, the **Proposed-Wallace (PW)** and Dadda trees [8]. PW and Dadda have been empirically shown to minimize and maximize the number of full adders required in the final adder chain, respectively. Both of these trees use only full and half adders as compressors, for which the Boolean relationships between their inputs and outputs are well known. Whenever the algorithm requires the use of a compressor, it is inserted as its logically equivalent combinational logic, constructed with Boolean gates. The final compression logic is thus a combinational circuit that can be optimized as a whole and then synthesized into LUTs by ABC.

CAD Improvement Validation. After implementing the proposed synthesis algorithms and integrating them into VTR, we validated the changes using the Kratos benchmark suite. Figure 5 summarizes the geometric mean of adder usage, ALM usage, critical path delay, and ADP across

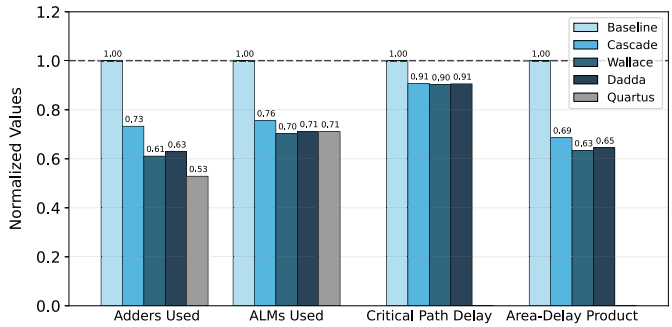


Fig. 5. Normalized adders, ALMs, critical path delay, and ADP comparison between baseline VTR and our improved Cascade, Wallace, and Dadda adder synthesis algorithms on the Kratos benchmark set. Quartus area results are also displayed for comparison.

various benchmarks and operand widths on the Stratix 10 FPGA. For reference, we also include the results of adder and ALM utilization from Intel Quartus Prime Pro 23.1 to compare resource efficiency with a commercial-grade tool. The goal is not to outperform Quartus, which uses more advanced optimizations that we have not implemented. Instead, it is to validate whether our CAD flow generates realistic FPGA resource utilization and is representative of a commercial FPGA CAD flow. We omit Quartus timing results as it uses a different timing model, and the open source Stratix-10-like architecture in VTR does not provide accurate timing data. Overall, our updates to VTR improve upon the baseline by ~37% in terms of ADP and closely match Quartus in terms of resource utilization. This provides a solid foundation for fair and realistic comparisons in our end-to-end evaluations presented in the next sections.

5 Evaluation

5.1 Experimental Setup

VTR Setup. The baseline FPGA architecture used for evaluation is modeled after the Intel/Altera Stratix 10 architecture, with a 20-nm technology node for delay and area estimation. This model is directly available from the VTR repository and was developed in prior work [18]. We build upon this Stratix-10-like architecture by modeling the AddMux, AddMux Crossbar, and different output multiplexing to get the DD5 and DD6 architectures. Timing models for these components are obtained from COFFE 2. The number of ALMs per LB is fixed at 10, as in the original architecture, and all other FPGA components remain unchanged. We fix the channel width at 400, and we set the VTR `target_ext_pin_util` to 0.9 for both inputs and outputs—this allows the VTR packer to use up to 90% of the inputs and outputs of a LB. The adder synthesis algorithm is set to use "Wallace" as this gives the best results in all metrics, as shown in Figure 5. The placement and routing are all timing-driven, and we run every experiment three times with three different seeds and take the average to get representative results.

Benchmark Selection. To assess the impact of our architectural modifications, we evaluate our designs across the three benchmark suites shown below. Table 3 also summarizes key information about each benchmark.

—VTR Standard: A general-purpose FPGA benchmark suite from the VTR repository [37]. This benchmark set ensures that our modifications do not degrade the versatility of the FPGA in broader application scenarios.

Table 3. Key Statistics of Our Three Benchmark Suites, Measured on the Baseline Stratix 10 Architecture with VTR

Benchmark	Num. Circuits	ALMs ($\times 10^3$)		Adder Percent		Avg. Fmax (MHz)
		Avg.	Max	Avg.	Max	
VTR [37]	19	10.2	59.7	19.5%	47.7%	109.5
Koios [6]	20	64.3	360.7	22.5%	50.2%	70.9
Kratos [16]	7	59.6	208.2	61.4%	70.7%	103.7

—Koios: A benchmark suite focusing on machine learning designs implemented on FPGAs [6]. This benchmark set helps to assess how well our architecture adapts to typical ML workloads beyond Kratos.
—Kratos: A specialized set of circuits designed for unrolled DNN inference [16]. Kratos allows customized circuit sizes and varying computation sparsity levels, making it useful for studying our optimizations under high soft-arithmetic pressure. Because Kratos is intentionally arithmetic-heavy, we report it alongside Koios and VTR rather than treating it as representative of all FPGA workloads. In this evaluation, we used the same "small-size" set of benchmarks as in the original Kratos paper.

5.2 Experiment Results

Area and Delay Evaluation. Figure 6 presents the normalized results of our proposed DD5 architecture compared to the baseline Stratix 10 architecture across the Koios, VTR, and Kratos benchmark suites. In terms of ALM area usage, DD5 consistently reduces resource consumption, achieving an average reduction of 10.9%. This improvement is particularly substantial in the Kratos benchmark suite, where arithmetic operations dominate, showing an average of 21.6% area savings. While the average critical path delay remains close to the baseline, a few circuits do exhibit an increase of up to 16%. These cases are likely due to suboptimal packing, increased routing congestion, and routing heuristics that are not yet optimized for our modified architecture. It is also important to note that the extra Double-Duty paths are fixed architectural resources rather than runtime controls: a mapped design may still use an ALM purely as LUT logic, but the hardware cannot disable the added muxing after fabrication to exactly recover the baseline critical path. Lastly, when considering the ADP, DD5 demonstrates an average improvement of 9.7%, indicating the effectiveness of our new Double-Duty Architecture.

DD5 vs. DD6. We additionally evaluate the second variant DD6, which allows concurrent and independent usage of 6-LUT and adders, and should bring more flexibility compared to DD5. Figure 7 shows the comparison between DD5 and DD6. While DD6 provides minor additional area savings in Kratos benchmarks, no noticeable gains are observed in Koios or VTR benchmarks. This can be attributed to two factors: (1) the current CAD tools are not optimized for architectures supporting concurrent 6-LUT usage and (2) the utilization of 6-LUTs is inherently low. On average, only 7% of ALMs across these benchmarks make use of 6-LUTs, making the opportunity for concurrent usage limited. This low 6-LUT utilization also suggests a simpler design choice: rather than paying the full DD6 output-mux cost, future variants could consider repurposing some rarely used 6-LUT input flexibility for Z-input access. Additionally, the added complexity of the output multiplexers in DD6 leads to higher critical path delays, with an average frequency penalty of approximately 8%. Consequently, the ADP also increases. This tradeoff is not worthwhile for the evaluated workloads, and it reinforces our choice to use DD5 as the main design point.

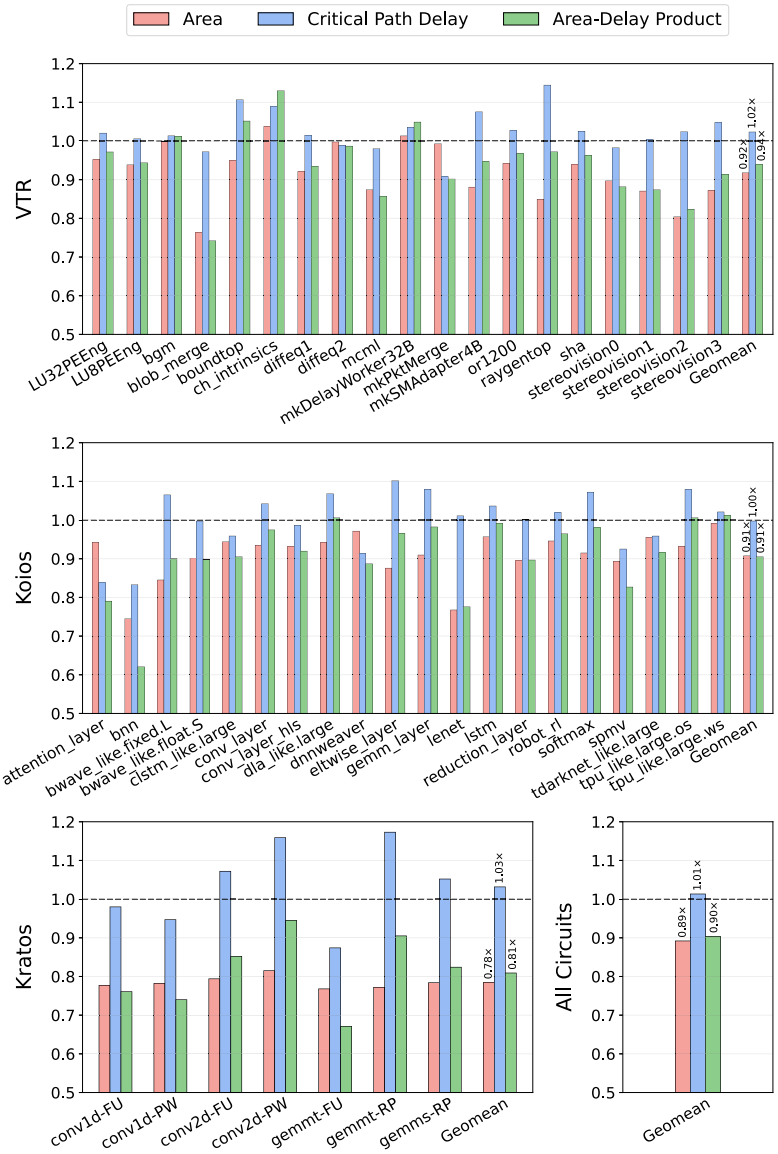


Fig. 6. Normalized ALM area usage, critical path delay, and ADP of Double-Duty DD5 Architecture, evaluated on the Koios, VTR, and Kratos benchmarks.

Routing Congestion. To analyze the routing impact of the proposed DD5 architecture, we examined the distribution of routing channel utilization using the Kratos benchmark set, which showed the largest area saving (22%) and the highest critical path delay penalty (3%) on average. The result is shown in Figure 8. While the portion of underutilized channels slightly increased, the majority of the channel utilization shifted toward a higher range, especially the portion between 0.3 and 0.6. This pattern suggests that by packing the same amount of logic into fewer LBs, routing channels become more congested, leading to denser usage. For all the circuits in these benchmarks, none failed to route, and the average critical path delay impact is modest. This suggests that the added

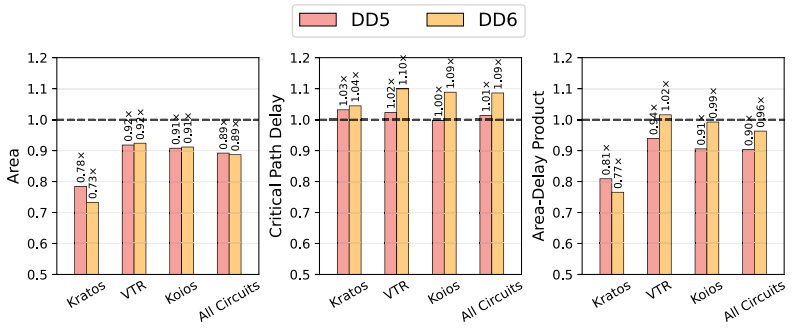


Fig. 7. Normalized ALM area used, critical path delay, and ADP comparison between DD5 and DD6 architectures on the geometric means of VTR, Koios, and Kratos benchmarks.

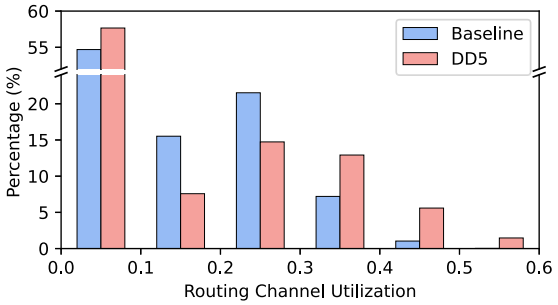


Fig. 8. Histogram of routing channel utilization averaged over the Kratos benchmark set. DD5 increases routing congestion on average as the diagram shows a shift of the histogram to higher channel utilization ranges.

congestion is manageable for the modeled routing fabric at the fixed channel width used in our experiments, although it does not imply that routing resources are free or that every commercial routing fabric would have the same margin.

Packing Stress Test. To quantify the limits of concurrent 5-LUT usage in DD5, we performed an *artificial* packing stress test. In this test, we constructed a synthetic circuit with 500 adders and incrementally added 5-LUT logic to the circuit, up to 500 LUTs in total. Under perfect concurrency conditions, this would be the maximum number of LUTs that could be absorbed into ALMs, concurrently with adders, without increasing utilization. We enabled the "allow unrelated clustering" option in VPR to encourage maximum packing density at the cost of ignoring timing, hence the "artificial" nature of this test. Figure 9 shows the results. The left plot presents total area usage in **Minimum-Width Transistor Areas (MWTAs)**. When only adders are present, the baseline architecture has a slight area advantage since the DD5 architecture has a small area overhead in each ALM. However, when LUTs are added, the DD5 architecture shows a significant advantage as it can absorb LUTs into existing ALMs with adders, keeping the area the same until the ALMs are saturated. The right plot shows the number of concurrently packed 5-LUTs, which saturates at 375, corresponding to 75% of the theoretical maximum. This indicates that up to 75% concurrency between LUTs and adders is achievable before becoming constrained by other factors like the global and local interconnect.

End-to-End Stress Test. In this *realistic* stress test, we aim to evaluate how effectively the Double-Duty architecture improves packing density under constrained FPGA resources. The procedure

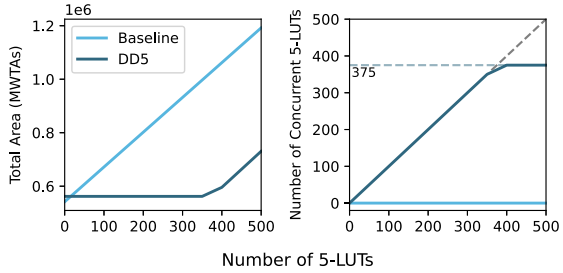


Fig. 9. Results of the packing stress test. It starts with a circuit of 500 adders, and we pack an increasing number of LUTs into the circuit. The *x*-axis is the number of LUTs packed, which saturates at 375 concurrent LUTs.

Table 4. Results of the End-to-End Stress Tests

Base Kratos Circuit	**conv1d-FU-mini**			**conv2d-FU-mini**			**gemmt-FU-mini**		
Architecture	**Base**	**DD5**	**Δ%**	**Base**	**DD5**	**Δ%**	**Base**	**DD5**	**Δ%**
Maximum SHA instances	5	9	+80.0%	3	5	+66.7%	11	13	+18.2%
Adders ($\times 10^3$)	37.55	38.79	+3.29%	18.86	19.48	+3.28%	14.35	14.97	+4.31%
5-LUTs ($\times 10^3$)	9.21	15.90	+72.5%	5.30	8.82	+66.5%	19.74	23.54	+19.2%
Concurrent 5-LUTs	0	4,397	+27.7%[a]	0	2,458	+27.9%[a]	0	3,790	+16.1%[a]
Critical Path Delay (ns)	15.81	15.05	−4.81%	16.07	15.01	−6.60%	16.27	15.16	−6.82%
ALM Count ($\times 10^3$)	27.93	27.08	−3.04%	14.16	13.79	−2.64%	18.45	18.04	−2.27%
LB count	2,947	2,946	−0.03%	1,474	1,498	+1.63%	1,895	1,870	−1.32%
Total ALM area (MWTAs $\times 10^6$)	60.54	60.88	+0.57%	30.69	30.99	+0.98%	40.00	40.55	+1.37%

The table compares the maximum number of additional SHA instances packed into a fixed-size FPGA for three Kratos circuits using both the baseline and the DD5 architecture. The DD5 architecture enables significant improvements in packing density and slight reductions in critical path delay.
[a]The percentage of 5-LUTs in which LUTs and adders are used concurrently. This is impossible in the baseline architecture.

is as follows: we implement a circuit from the Kratos benchmarks to determine the FPGA size needed for a successful implementation. Then, keeping this FPGA size fixed, we incrementally add instances of a small SHA circuit from the VTR standard benchmark suite until VTR can no longer complete placement and routing. This mirrors an internal industrial stress test in industry [22]. We conducted this experiment using both the baseline architecture and the DD5 architecture on three different Kratos circuits, and all experiments were performed with timing-driven placement and routing to better reflect a realistic scenario. Table 4 shows the results, indicating that DD5 enables significant improvements in packing density. For instance, the maximum number of SHA instances that can be implemented in the same FPGA area increased by 80% for `conv1d-FU-mini` and by 66.7% for `conv2d-FU-mini`, compared to the baseline. This is achieved through significant 5-LUT concurrency: 27.7% for `conv1d-FU-mini`, 27.8% for `conv2d-FU-mini`, and 16.1% for `gemmt-FU-mini`. Additionally, the critical path delay also shows slight improvements; this is because the critical path lies in the multiple-adder region, and our AddMux in the DD5 architecture reduced the delay from the ALM input to the adder, resulting in a lower overall critical path delay. These results demonstrate that DD5 not only allows for denser packing but can also improve timing in certain stress scenarios.

Input Sharing Ablation. One key tradeoff of the Double-Duty architecture is a 3.72% per-ALM area increase, primarily due to the AddMux crossbar required to expand the number of ALM inputs

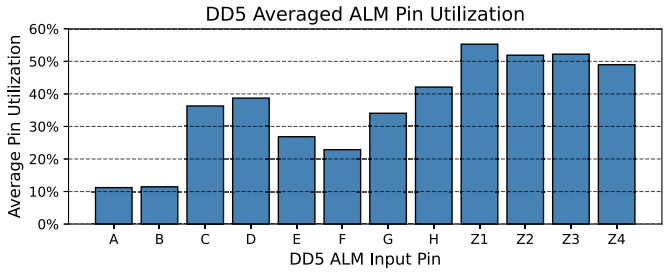


Fig. 10. Averaged input pin utilization for DD5 ALMs on the Kratos benchmark set. The A and B inputs are used the least by far, followed by the E and F inputs.

from 8 to 12. While the DD5 architecture can theoretically utilize all 12 ALM inputs simultaneously, practical usage is constrained by LB input availability. Each Stratix-10 LAB provides 60 inputs, evenly distributed across 10 ALMs; this limits average effective usage to 6 inputs per ALM. Across the Kratos benchmark suite, fewer than 2.5% of utilized ALMs require more than 6 inputs, indicating substantial space to reduce ALM input capacity without impacting workloads. Leveraging this pin-limited behavior, we explore sharing AddMux inputs with standard Stratix-10 ALM pins to reduce area.

Shared pins may either drive an AddMux or perform their original function, and each shared input allows a corresponding portion of the AddMux crossbar to be removed. We keep the general structure of the crossbar the same, connecting to 10 out of 60 LAB inputs (17% populated overall). However, each shared input reduces the crossbar's area since fewer connections have to be made between each ALM and the LAB inputs. We designed and evaluated four different input sharing architectures, successively removing each direct path from the crossbar to the adder chain. The final implementation shares all four direct AddMux inputs with standard Stratix-10 ALM inputs, removing the AddMux Crossbar entirely. Although increased sharing reduces connection flexibility and limits simultaneous LUT and adder usage in certain cases, the resulting loss is outweighed by area savings for several configurations.

Input pin selection for sharing is guided by empirical utilization: Figure 10 shows average ALM input usage across Kratos benchmarks, where pins[2] A and B are least used (11.2% and 11.4%), followed by E and F (16.8% and 22.9%). Accordingly, we assign A for Share 1; A and B for Share 2; A, B, and E for Share 3; and A, B, E, and F for Share 4. These pins replace the dedicated Z1, Z2, Z3, and Z4 inputs. Using the same COFFE 2 methodology as Section 3.2, we create SPICE circuit models for each of the sharing architectures. We optimize for area × delay. The vast majority of the ALM stays the same, except for changes to the AddMux crossbar, which has the same structure but now connects to a smaller number of direct inputs. Sharing inputs slightly changes transistor sizing within the ALM, but the changes in area are tiny compared to the AddMux Crossbar itself. The AddMux Crossbar area drops almost linearly from 77.91 MWTA for the DD5 (Share 0) architecture to 55.54, 36.97, and 18.48 MWTA for the Shares 1–3 variants, and drops to 0 for Share 4.

Figure 11 presents normalized results for each input sharing architecture relative to both the DD5 and the baseline Stratix-10 architecture, evaluated on the Kratos benchmark suite with the same experimental setup described in Section 5.1. Overall, all sharing variants exhibit ADPs that are very close to the base DD5 design, indicating that input sharing does not substantially degrade overall efficiency. In particular, Shares 1–4 reduce area by 2.56%, 1.28%, 5.13%, and 2.56%, respectively,

[2]Please refer to Figure 2 for pin naming.

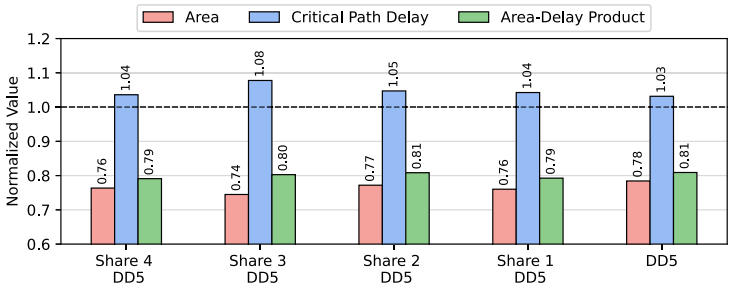


Fig. 11. Sharing architectures evaluated on selected Kratos circuits and normalized with respect to baseline Stratix-10. Sharing architectures and DD5 perform similarly for all experiments.

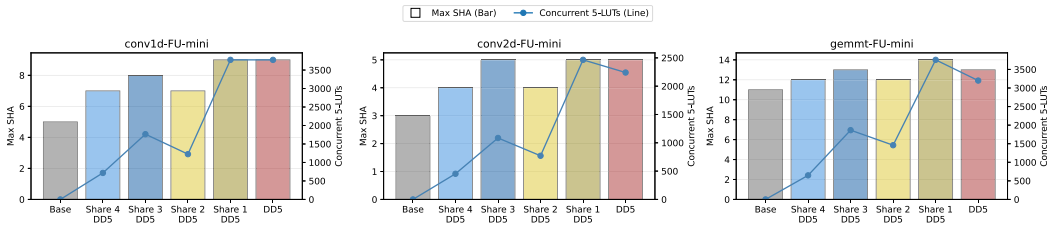


Fig. 12. End-to-End stress tests for sharing architectures. The plots compare the maximum number of additional SHA instances packed into a fixed-size FPGA for three Kratos circuits across the sharing architectures. The methodology follows from Table 4.

compared to the base DD5. All of the sharing architectures balance the area improvements with increased critical path delay, meaning that the ADP is roughly the same as DD5.

However, Shares 1 and 4 improve their ADP by 2.47% compared to DD5. This indicates that the decrease in routing flexibility does not outweigh the area benefits for Shares 1 and 4, while for the other sharing architectures, the two cancel out almost exactly. The similar results for DD5 and the sharing architectures indicate that these variants are largely interchangeable on the evaluated circuits, allowing designers to tradeoff area, delay, and routing flexibility.

We also subject the sharing architectures to end-to-end stress testing, as described before, to measure whether the packing flexibility is reduced as more inputs are shared. Figure 12 shows the results, with the packing ability decreasing as the number of shared inputs increases. This can also be seen in the number of concurrent 5-LUTs dropping as the number of possible routes into each ALM shrinks. Notably, Share 1 performs better than DD5, packing one more SHA for both the conv1d-FU-mini and gemmt-FU-mini benchmarks with similar or more concurrent LUT5s. This comes as a result of the shared inputs coming from the Local Crossbar instead of the much more sparsely populated AddMux Crossbar, since there are now more possible routes directly into the AddMux. This increased routing flexibility can allow for denser packing for Share 1, but is overshadowed by the effects of removing dedicated direct inputs for the other sharing architectures. Figure 12 shows that Shares 2 and 3 maintain relatively dense packing compared to the base Stratix-10 architecture; however, fewer LUT5s are packed concurrently with adders due to the lower ALM input flexibility. Overall, the results indicate that at least one input pin can be shared with existing ALM inputs, with the potential for sharing additional pins if LUT5 concurrency doesn't affect overall packing density.

6 Comparison and Compatibility with Prior Architectures

In this extended section, we investigate the compatibility of DD5 with prior FPGA architectural optimizations for arithmetic operations [9, 18]. The section is organized as follows. Section 6.1 briefly reviews the relevant prior work and its architectural designs. Section 6.2 describes how DD5 is integrated with these existing optimizations. Section 6.3 details the circuit-level modeling of the resulting fused architecture. Section 6.4 presents the additional CAD-tool optimizations required to enable efficient packing on the fused architecture. Finally, Section 6.5 reports the experimental results of several benchmarks on the new fused architecture.

6.1 Existing Architecture Designs

Prior work [9] has explored modifications to the FPGA architecture, motivated by the inefficiency of mapping low-precision arithmetic onto conventional ALM structures. In baseline Stratix-10-style ALMs, LUTs are underutilized when implementing multipliers: the LUTs generate partial products, while the carry chain performs most of the reduction, leaving logic resources idle in later reduction stages. To address this imbalance, they proposed a number of modifications to the ALM that compete and complement Double-Duty.

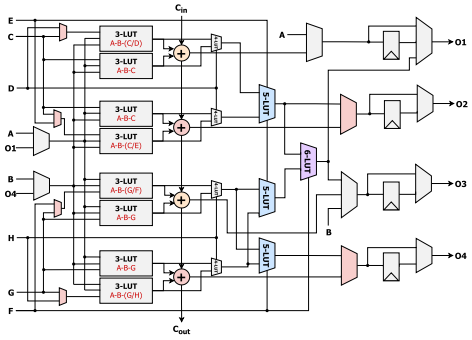
Four-Bit Adders and Deeper Fracturability. One approach [9, 18] proposes increasing arithmetic density by further fracturing each 4-LUT into two 3-LUTs and increasing the number of hard adders per ALM from two to four, forming a more compact carry chain. This design allows an ALM to generate and reduce more partial products, improving multiplier and **Multiply-Accumulate (MAC)** density, exploiting the high input sharing inherent in multiplier arrays. Two distinct versions of the 4-bit adder architecture are proposed in Figure 13, which differ in how carry propagation is distributed across ALMs:

- **Four-Bit Single Adder (4BSA)**. A single carry chain traverses only the first five ALMs in the LB. The remaining five ALMs do not have a carry chain passing through their adders. This architecture still allows concurrent LUT and adder usage *at the LB level*. Half the ALMs can be used for arithmetic, while the other half can be used for general LUT-based logic.
- **Four-Bit Double Adder (4BDA)**. Two carry chains are provided. The first traverses the initial five ALMs, while the second propagates through the remaining five ALMs, enabling arithmetic functionality across the full LB. Note that the LB inputs still only have 60 input pins, so it is impossible to fully utilize all 40 adders with independent inputs in this architecture because 80 inputs would be needed in that case.

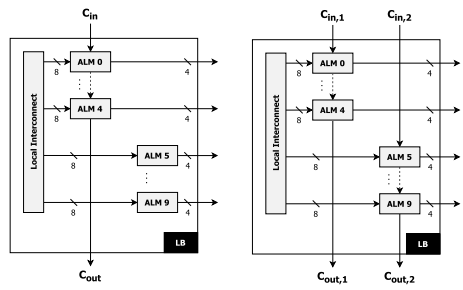
Extra Carry Chain (ECC). Another approach [18] introduces an additional carry chain within each ALM, enabling multiple stages of addition to be performed locally as shown in Figure 14(a). In this design, the sum outputs of the original carry chain feed a second carry chain that also consumes otherwise unused ALM inputs. This allows two levels of reduction to be executed within the same ALM, rather than spilling into subsequent ALMs as in the baseline architecture. As a result, partial product reduction for multipliers and accumulators can be compressed to just one adder chain, reducing the number of ALMs required and inter-ALM routing. The ECC architecture also effectively uses three inputs for every two adders because of the cascade from the first adder chain to the second adder chain. This means that 60 unique inputs would be needed to fully utilize the 2 chains, matching the LAB input restrictions.

6.2 Double-Duty with ECC

We combine ECC with Double-Duty and propose two new variations: the first is *ECC-DD5* shown in Figure 14(b), where four additional direct inputs with four multiplexers that feed into the first

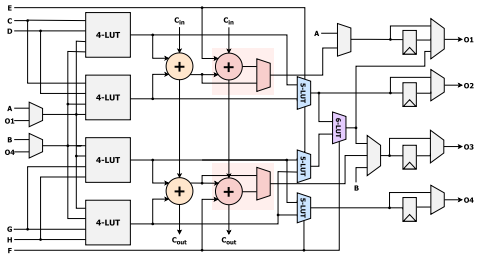


(a) ALM Design of 4-bit adder architectures.

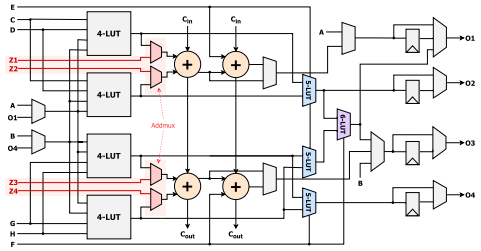


(b) LAB adder chain connections for the 4-bit single adder architecture (4BSA, left) and the 4-bit double adder architecture (4BDA, right).

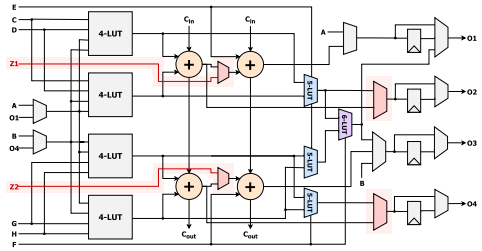
Fig. 13. Four-bit adder architecture designs [18].



(a) Extra Carry Chain Architecture [18].



(b) ECC with Double-Duty extension. (ECC-DD5)



(c) ECC with concurrent single adder chains. (ECC-SkipAdd)

Fig. 14. ECC architectures and proposed modifications.

adder chain. Similar to the DD5 design, this enables concurrent usage of the LUTs and adder chain. However, it is important to note that there are shared inputs (E and F) between each 5-LUT and its corresponding adders, limiting 5-LUT concurrency.

The second design, *ECC-SkipAdd* shown in Figure 14(c), adds two direct inputs and multiplexes them with the first adder chain's sum-output wires before they enter the second adder chain. In addition, the sum bits of the first adder chain are multiplexed with the 5-LUT outputs O2 and O4, allowing all adder chains to output on different ALM outputs instead of sharing O1 and O3. ECC-SkipAdd shares the first adder chain inputs with existing ALM inputs as shown by our ablation in Section 5. This architecture focuses on providing the maximum flexibility to the adder chains.

Table 5. Area (Per ALM) of Each Architecture Variant, Normalized against the Baseline Stratix-10 Architecture

Architecture	Baseline	DD5	4BSA, 4BDA	ECC	ECC-DD5	ECC-SkipAdd
Area (MWTAs)	2,167.3	2,248.0	2,193.9	2,199.9	2,282.9	2,208.1
Δ vs. Baseline	–	+3.72%	+1.23%	+1.50%	+5.33%	+3.38%

6.3 Circuit-Level Modeling

We use COFFE 2 to obtain area and delay values of the existing 4-bit adder (4BSA and 4BDA) and ECC architectures. While COFFE modeling was done on these architectures in the prior work [9], they optimized for area × delay squared with different FPGA parameters as well, so we redesign these architectures using the same parameters as performed in Section 3.2. We also simulate our ECC-DD5 and ECC-SkipAdd architectures, creating a SPICE model for the multiplexer into the second adder chain as well as using the same AddMux population. Table 5 contains the measured ALM areas for all new architectures.

6.4 Additional VTR Optimizations

To fully exploit ternary adder chain architectures, the CAD flow must be modified to generate arithmetic structures that map efficiently onto ECC-style carry chains. Two complementary enhancements were introduced and validated.

ECC-Friendly Synthesis. The Wallace compressor tree iteratively reduces operands until only two remain, at which point a final carry propagate adder is used. This behavior is optimized for binary adder chains, but suboptimal for the ECC architectures. We therefore introduce *Wallace–Ternary*, a simple modification to the algorithm that stops compression when three operands remain instead of two, mapping directly onto ECC adder chains.

To validate the enhancement, we test the synthesis modes Cascade, Wallace, and Wallace–Ternary on the ECC architecture on mini Kratos benchmarks. The results can be found in Figure 15. We see that while the number of adders using Wallace–Ternary synthesis increases slightly, the number of ALMs used is slightly decreased. This makes sense, because more adders are included due to fewer reduction operations; however, this enables fuller utilization of the cascaded adder chain. Importantly, the critical path delay decreases from standard Wallace synthesis by 15.1%, a result of fewer reduction stages that need to be implemented as LUT logic.

Kratos Tree-Based Adder Reduction. The Kratos benchmark suite contains large MAC graphs composed of dense partial product reduction trees. By default, Kratos emits binary adder trees, where each node reduces exactly two operands. This synergizes well with the Stratix-10 architectures, as the two-operand additions are generally synthesized into single adder chain instances. However, this structure prevents full utilization of the ECC, which implements ternary reduction, hence resulting in the second row of adders being underutilized. To address this, we modify the Kratos arithmetic synthesis to emit base-3 reduction trees instead of two.

To validate the enhancement, we generalize Kratos reduction trees to support a configurable base B. Using the base Stratix-10, 4BDA, and ECC architectures, we test different values of B on a Kratos subset and plot the area usage and delay. For the ECC architecture, we use Wallace–Ternary, and for the base and 4BDA we use Wallace for adder synthesis. Figure 16 shows that the ECC architecture has the greatest area reduction by 28% from $B = 2$ to $B = 3$. Moreover, while the single carry chain architectures experience an increase in critical path delay, the critical path delay for ECC *decreases*. Hence, the base $B = 2$ is optimal for *binary* adder chain variants (Stratix 10, Double-Duty, 4BSA, and 4BDA), and $B = 3$ is optimal for *ternary* adder chain variants (ECC, ECC-DD5, and ECC-SkipAdd).

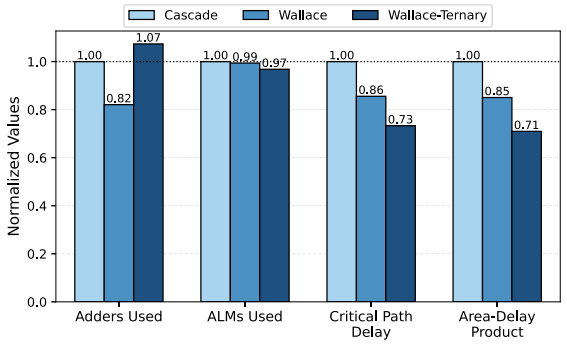


Fig. 15. Normalized adders, ALMs, critical path delay, and ADP comparison between Cascade, Wallace, and Wallace–Ternary on the ECC architecture. While the number of adders increases due to less reduction, the number of ALMs stays comparable while decreasing the CPD compared to the existing methods.

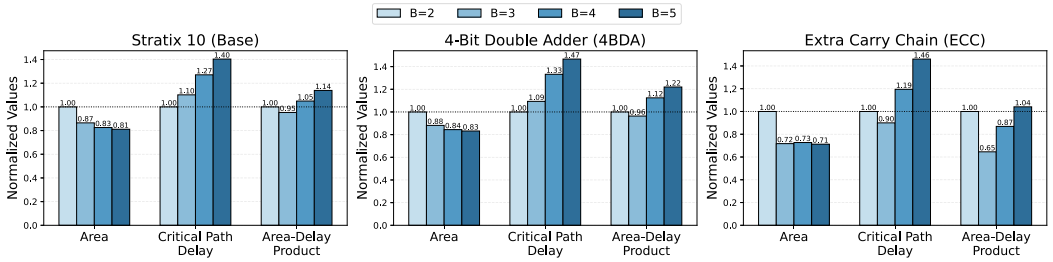


Fig. 16. Different tree bases on adder reduction on the Kratos benchmarks. Architectures with single carry chains do not benefit from higher tree bases, while the ECC architecture benefits at $B = 3$.

6.5 Experiments

We follow the same experimental setup as outlined in Section 5.1 with slight modifications. The ECC-based architectures—ECC, ECC-DD5, and ECC-SkipAdd—use the adder synthesis algorithm Wallace–Ternary and use a tree base $B = 3$ for Kratos benchmarks as outlined in the prior subsection.

Comparison with 4-Bit Adders and ECC. Figure 17 summarizes the normalized results for the modified carry-chain architectures, including the proposed ECC-DD5 and ECC-SkipAdd designs. The 4-bit adder architectures (4BSA and 4BDA) reduce area by 18% and critical path delay by 3% relative to the baseline architecture, yielding a 21% improvement in ADP. When compared against DD5, these designs achieve a 12% ADP improvement. The ECC architecture achieves a 13% reduction in area and a 9% reduction in critical path delay, corresponding to a 20% improvement in ADP relative to the baseline and an 11.1% improvement over DD5. The performance gains from the 4-bit adder architectures are consistent across all benchmark suites. In contrast, ECC exhibits its strongest improvements on the Kratos benchmarks, where ADP is reduced by up to 52%, while providing modest gains on the VTR and Koios benchmarks. This behavior is expected, as the Kratos workloads are the most arithmetic-intensive and therefore benefit most from the additional carry-chain resources, whereas VTR and Koios represent more general-purpose designs.

ECC+DD5 Variants. Combining the Double-Duty design with the ECC architecture does not further improve performance on average; the ADP of ECC-DD5 remains the same as ECC. Breaking down performance by benchmark, the ADP of ECC-DD5 is unchanged on VTR and Koios. On the Kratos benchmark, we observe a noticeable decrease in area by 14.8% and an increase in critical path delay by 8.9% relative to ECC, overall providing an improvement of 6% in ADP. This

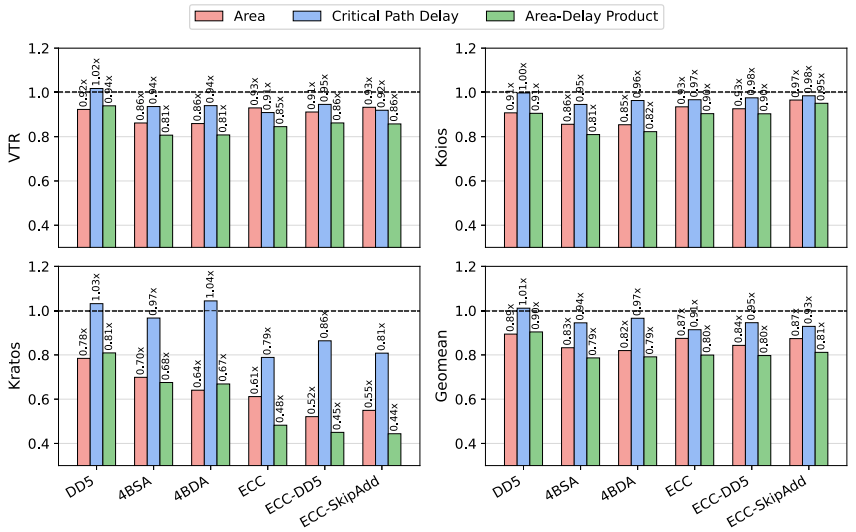


Fig. 17. Area, critical path delay, and ADP on VTR, Koios, and Kratos benchmarks for the different architecture variants.

Table 6. Results of the End-to-End Stress Tests for 4-Bit and Extra-Carry-Chain-Based Architectures

	Base	DD5	DD5 (Wallace)	4BSA	4BDA	ECC	ECC-DD5	ECC-SkipAdd
conv1d-FU-mini	5	9	11	11	14	15	16	16
conv2d-FU-mini	3	5	6	5	7	7	8	8
gemmt-FU-mini	11	13	14	13	14	14	15	15

The numbers in the table represent the maximum number of SHA circuits that can be successfully placed and routed with each Kratos base circuit in a constrained FPGA size (similar to Table 4). We additionally report DD5 with Wallace adder synthesis; it improves over the prior DD5 results obtained with cascade synthesis because more LUT-based reductions are available for concurrent packing.

architecture variant provides the densest packing of the Kratos benchmark circuits thus far, by enabling concurrent LUT usage with dual carry chains, giving maximum flexibility to VTR to map logic and arithmetic primitives onto the FPGA. ECC-SkipAdd provided denser packing compared to ECC, but generally performed poorly in many settings, with noticeably worse performance on Koios relative to ECC.

End-to-End Stress Test. We run stress test experiments using the methodology outlined in Section 5.2 on the 4-bit and ECC-based architectures. In this experiment, we use the Wallace compressor tree mode on base and DD5 as well to provide a fair comparison with the new architectures. As shown in Table 6, across all architectures, we observe denser packing relative to the baseline. Among the 4-bit adder variants, 4BSA performs slightly worse than DD5, packing one fewer SHA on both `conv2d-FU-mini` and `gemmt-FU-mini`, while 4BDA consistently outperforms DD5, achieving an increase of three SHAs on `conv1d-FU-mini` and one additional SHA on `conv2d-FU-mini`. The ECC architecture further improves packing, exceeding DD5 by four SHAs on `conv1d-FU-mini` and by one SHA on `conv2d-FU-mini`. Finally, both ECC-DD5 and ECC-SkipAdd perform best by packing an additional SHA on all Kratos circuits over ECC.

Compatibility with Other Architectures. Our architectural exploration and evaluation are based on an Intel/Altera Stratix-10-like FPGA, as its open source architectural models are readily available

[18]. However, modern FPGA families—such as the AMD Versal series [2]—feature different design paradigms: they use LUTs as full adders and include dedicated carry-look ahead logic, fundamentally different from the Stratix 10. To adapt the Double-Duty architecture to these FPGAs, several function unit augmentations would be necessary, including reintroducing explicit 1-bit full adders. While this requires some additional logic, the relative area cost of full adders in modern technologies is negligible and unlikely to cause substantial overhead. As a result, we believe our Double-Duty architecture is adaptable to modern devices with minimal design changes and without compromising its core efficiency benefits. A related academic architecture, LUXOR [36], achieves higher arithmetic efficiency of compressor trees by improving the expressiveness of each ALM through an extra XOR gate. This approach is also orthogonal to ours, as LUXOR focuses on gate-level function enhancement, whereas Double-Duty focuses on improving data path flexibility and resource utilization. Therefore, both enhancements can be applied to the same FPGA simultaneously. We plan to more accurately compare with these alternative LB architectures in future work.

7 Conclusion

We presented Double-Duty, a novel FPGA LB architecture that enables the concurrent and independent usage of adders and LUTs within the same ALM, significantly improving arithmetic density with minimal area overhead and a negligible impact on critical path delay. Together with the architectural innovations, we enhanced the VTR CAD toolchain [19] with optimized synthesis algorithms for adder chains, substantially improving its efficiency and providing us with a baseline that is representative of commercial CAD tools like Quartus. We evaluated Double-Duty across 3 diverse benchmark suites, achieving an average 9.7% improvement in ADP and up to 21.6% area savings on the arithmetic-heavy Kratos benchmarks [16]. Although denser packing may introduce routing pressure, we found the increase in routing congestion to be manageable in our modeled routing fabric and fixed-channel-width experiments. Additionally, two stress tests show the potential of Double-Duty in achieving higher density and resource utilization. Finally, we implemented architectures from prior work within VTR [18]—specifically, 4-bit-per-ALM adder architectures and dual carry chain architectures—to be able to compare them to and combine them with Double-Duty architectures. The fused architecture achieved a 20% decrease in ADP on the geometric mean across multiple benchmarks. Our results demonstrate that Double-Duty is compatible with other architectural optimizations and can achieve higher area density and better ADP. While some modern FPGAs [2] adopt different base architectures, we believe the design methodology and principles behind Double-Duty are broadly applicable and can inspire future developments in arithmetic-centric FPGA LB design.

Acknowledgment

We would like to thank Sergey Gribok, Ilya Ganusov, Martin Langhammer, Sadegh Yazdanshenas, and Susanne Balle for discussion and feedback.

References

[1] Mohamed S. Abdelfattah, David Han, Andrew Bitar, Roberto DiCecco, Shane O'Connell, Nitika Shanker, Joseph Chu, Ian Prins, Joshua Fender, Andrew C. Ling, et al. 2018. DLA: Compiler and FPGA overlay for neural network inference acceleration. In *Proceedings of the 29th International Conference on Field Programmable Logic and Applications (FPL)*.

[2] AMD. 2024. Versal Adaptive SoC Configurable Logic Block Architecture Manual (AM005). Retrieved from https://docs.amd.com/r/en-US/am005-versal-clb/CLB-Architecture

[3] AMD. 2025. Versal Adaptive SoC Technical Reference Manual (AM011) Digital Signal Processor. Retrieved from https://docs.amd.com/r/en-US/am011-versal-acap-trm/Digital-Signal-Processor

[4] Marta Andronic and George A. Constantinides. 2023. PolyLUT: Learning piecewise polynomials for ultra-low latency FPGA LUT-based inference. In *Proceedings of the 2023 International Conference on Field Programmable Technology (ICFPT)*, 60–68. DOI: https://doi.org/10.1109/ICFPT59805.2023.00012

[5] Aman Arora, Atharva Bhamburkar, Aatman Borda, Tanmay Anand, Rishabh Sehgal, Bagus Hanindhito, Pierre-Emmanuel Gaillardon, Jaydeep Kulkarni, and Lizy K. John. 2022. CoMeFa: Deploying compute-in-memory on FPGAs for deep learning acceleration. In *Proceedings of the 30th IEEE Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*.

[6] Aman Arora, Andrew Boutros, Seyed Alireza Damghani, Karan Mathur, Vedant Mohanty, Tanmay Anand, Mohamed A. Elgammal, Kenneth B. Kent, Vaughn Betz, and Lizy K. John. 2023. Koios 2.0: Open-source deep learning benchmarks for FPGA architecture and CAD research. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 11 (2023), 3895–3909. DOI: https://doi.org/10.1109/TCAD.2023.3272582

[7] Aman Arora, Samidh Mehta, Vaughn Betz, and Lizy K. John. 2021. Tensor slices to the rescue: Supercharging ML acceleration on FPGAs. In *Proceedings of the 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*.

[8] Shahzad Asif and Yinan Kong. 2014. Low-area Wallace multiplier. *VLSI Design* 2014 (May 2014), 1–6. DOI: https://doi.org/10.1155/2014/343960

[9] Andrew Boutros, Mohamed Eldafrawy, Sadegh Yazdanshenas, and Vaughn Betz. 2019. Math doesn't have to be hard: Logic block architectures to enhance low-precision multiply-accumulate on FPGAs. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*.

[10] Andrew Boutros, Sadegh Yazdanshenas, and Vaughn Betz. 2018. Embracing diversity: Enhanced DSP blocks for low-precision deep learning on FPGAs. In *Proceedings of the 29th International Conference on Field Programmable Logic and Applications (FPL)*.

[11] Robert Brayton and Alan Mishchenko. 2010. ABC: an academic industrial-strength verification tool. In *Proceedings of the 22nd International Conference on Computer Aided Verification (CAV' 10)*. Springer-Verlag, Berlin, 24–40.

[12] Zachariah Carmichael, Hamed F. Langroudi, Char Khazanov, Jeffrey Lillie, John L. Gustafson, and Dhireesha Kudithipudi. 2019. Deep positron: A deep neural network using the posit number system. In *Proceedings of the 2019 Design, Automation and Test in Europe Conference and Exhibition (DATE)*, 1421–1426. DOI: https://doi.org/10.23919/DATE.2019.8715262

[13] Yuzong Chen and Mohamed S. Abdelfattah. 2023. BRAMAC: Compute-in-BRAM architectures for multiply-accumulate on FPGAs. In *Proceedings of the 31st IEEE Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*.

[14] Yuzong Chen, Jordan Dotzel, and Mohamed S. Abdelfattah. 2023. M4BRAM: Mixed-precision matrix-matrix multiplication in FPGA block RAMs. In *Proceedings of the 34th International Conference on Field Programmable Logic and Applications (FPL)*.

[15] Charles Chiasson and Vaughn Betz. 2013. COFFE: Fully-automated transistor sizing for FPGAs. In *Proceedings of the International Conference on Field-Programmable Technology (FPT)*, 34–41.

[16] Xilai Dai, Yuzong Chen, and Mohamed S. Abdelfattah. 2024. Kratos: An FPGA benchmark for unrolled DNNs with fine-grained sparsity and mixed precision. In *Proceedings of the 35th International Conference on Field Programmable Logic and Applications (FPL)*.

[17] J. Duarte, S. Han, P. Harris, S. Jindariani, E. Kreinar, B. Kreis, J. Ngadiuba, M. Pierini, R. Rivera, N. Tran, et al. 2018. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation* 13, 7 (2018), P07027. DOI: https://doi.org/10.1088/1748-0221/13/07/P07027

[18] Mohamed Eldafrawy, Andrew Boutros, Sadegh Yazdanshenas, and Vaughn Betz. 2020. FPGA logic block architectures for efficient deep learning inference. *ACM Transactions on Reconfigurable Technology and Systems* 13, 3, Article 12 (June 2020), 1–34. DOI: https://doi.org/10.1145/3393668

[19] Mohamed A. Elgammal, Amin Mohaghegh, Soheil Gholami Shahrouz, Fatemehsadat Mahmoudi, Fahrican Koşar,Kimia Talaei, Joshua Fife, Daniel Khadivi, Kevin Murray, Andrew Boutros, et al. 2025. VTR 9: Open-source CAD FOR fabric and beyond FPGA architecture exploration. *ACM Transactions on Reconfigurable Technology and Systems* 18, 3 (2025), 1–53. DOI: https://doi.org/10.1145/3734798

[20] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, Vivek Bhanu, Adrian Caulfield, Eric Chung, et al. 2018. Azure accelerated networking: SmartNICs in the public cloud. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[21] Brian Gaide, Dinesh Gaitonde, Chirag Ravishankar, and Trevor Bauer. 2019. Xilinx adaptive compute acceleration platform: Versal architecture. In *Proceedings of the 27th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*.

[22] Ilya Ganusov. 2025. Private Communication. Altera Corp., San Jose, CA.

[23] Mohamed W. Hassan, Hatem Ltaief, and Suhaib A. Fahmy. 2024. High throughput massive MIMO signal decoding using multi-level tree search on FPGAs. In *Proceedings of the 32nd IEEE Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*.

[24] Thijs Havinga, Xianjun Jiao, Wei Liu, and Ingrid Moerman. 2023. Accelerating FPGA-based WI-FI transceiver design and prototyping by high-level synthesis. In *Proceedings of the 31st IEEE Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*.

[25] Konstantin Hoßfeld, Hans Jakob Damsgaard, Jar Nurmi, Michaela Blott, and Thomas B. Preußer. 2024. High-efficiency compressor trees for latest AMD FPGAs. *ACM Transactions on Reconfigurable Technology and Systems* 17, 2, Article 30 (April 2024), 1–32. DOI: https://doi.org/10.1145/3645097

[26] Intel Corporation. 2017. Intel Arria Native Fixed Point DSP IP Core User Guide: Functional Description. Intel Corporation.

[27] Peter Jamieson, Kenneth B. Kent, Farnaz Gharibian, and Lesley Shannon. 2010. Odin II—An open-source Verilog HDL synthesis tool for CAD research. In *Proceedings of the IEEE Annual International Symposium on Field-Programmable Custom Computing Machines*, 149–156.

[28] Kimia Talaei Khoozani, Arash Ahmadian Dehkordi, and Vaughn Betz. 2023. Titan 2.0: Enabling open-source CAD evaluation with a modern architecture capture. In *Proceedings of the 34th International Conference on Field Programmable Logic and Applications (FPL)*.

[29] Martin Kumm and Peter Zipf. 2014. Pipelined compressor tree optimization using integer linear programming. In *Proceedings of the 2014 24th International Conference on Field Programmable Logic and Applications (FPL)*, 1–8. DOI: https://doi.org/10.1109/FPL.2014.6927468

[30] Martin Langhammer, Gregg Baeckler, and Sergey Gribok. 2019. Fractal synthesis: Invited tutorial. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '19)*. ACM, New York, NY, 202–211. DOI: https://doi.org/10.1145/3289602.3293927

[31] Martin Langhammer, Eriko Nurvitadhi, Bogdan Pasca, and Sergey Gribok. 2021. Stratix 10 NX architecture and applications. In *Proceedings of the 29th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*.

[32] Guoyu Li, Shaochuan Wu, Changsheng You, Wenbin Zhang, Guanyu Shang, and Xiaokang Zhou. 2023. Cell-free massive MIMO-OFDM: Asynchronous reception and performance analysis. *IEEE Internet of Things Journal* 11, 7 (2023), 11894–11906.

[33] Kevin E. Murray, Oleg Petelin, Sheng Zhong, Jia Min Wang, Mohamed Eldafrawy, Jean-Philippe Legault, Eugene Sha, Aaron G. Graham, Jean Wu, Matthew J. P. Walker, et al. 2020. VTR 8: High performance CAD and customizable FPGA architecture modelling. *ACM Transactions on Reconfigurable Technology and Systems* 13, 2 (2020), 1–55.

[34] Hadi Parandeh-Afshar, Philip Brisk, and Paolo Ienne. 2008. Efficient synthesis of compressor trees on FPGAs. In *Proceedings of the 2008 Asia and South Pacific Design Automation Conference*, 138–143. DOI: https://doi.org/10.1109/ASPDAC.2008.4483927

[35] Junius Pun, Xilai Dai, Grace Zgheib, Mahesh A. Iyer, Andrew Boutros, Vaughn Betz, and Mohamed S. Abdelfattah. 2025. Double duty: FPGA architecture to enable concurrent LUT and adder chain usage. In *Proceedings of the 36th International Conference on Field Programmable Logic and Applications (FPL)*.

[36] Seyedramin Rasoulinezhad, Siddhartha, Hao Zhou, Lingli Wang, David Boland, and Philip H. W. Leong. 2020. LUXOR: An FPGA logic cell architecture for efficient compressor tree implementations. In *Proceedings of the 28th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*.

[37] Jonathan Rose, Jason Luu, Chi Wai Yu, Opal Densmore, Jeffrey Goeders, Andrew Somerville, Kenneth B. Kent, Peter Jamieson, and Jason Anderson. 2012. The VTR project: Architecture and CAD for FPGAs from Verilog to routing. In *Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. ACM, New York, NY, 77–86. DOI: https://doi.org/10.1145/2145694.2145708

[38] Naif Tarafdar, Thomas Lin, Eric Fukuda, Hadi Bannazadeh, Alberto Leon-Garcia, and Paul Chow. 2017. Enabling flexible network FPGA clusters in a heterogeneous cloud data center. In *Proceedings of the 25th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*.

[39] Stephen Tridgell, Martin Kumm, Martin Hardieck, David Boland, Duncan Moss, Peter Zipf, and Philip H. W. Leong. 2019. Unrolling ternary neural networks. *ACM Transactions on Reconfigurable Technology and Systems* 12, 4, Article 22 (Oct. 2019), 1–23.

[40] Erwei Wang, James J. Davis, Peter Y. K. Cheung, and George A. Constantinides. 2019. LUTNet: Rethinking inference in FPGA soft logic. In *Proceedings of the IEEE International Symposium on Field-Programmable Custom Computing Machines*.

[41] Erwei Wang, James J. Davis, Georgios-Ilias Stavrou, Peter Y. K. Cheung, George A. Constantinides, and Mohamed Abdelfattah. 2022. Logic shrinkage: Learned FPGA netlist sparsity for efficient neural network inference. In *Proceedings of the 30th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*.

[42] Qinggang Wang, Long Zheng, Zhaozeng An, Shuyi Xiong, Runze Wang, Yu Huang, Pengcheng Yao, Xiaofei Liao, Hai Jin, and Jingling Xue. 2024. A scalable, efficient, and robust dynamic memory management library for HLS-based FPGAs. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*.
[43] Jagath Weerasinghe, Raphael Polig, Francois Abel, and Christoph Hagleitner. 2016. Network-attached FPGAs for data center applications. In *Proceedings of the 2016 International Conference on Field-Programmable Technology (FPT)*.
[44] Clifford Wolf. 2012. Yosys Open Synthesis Suite. Retrieved from https://yosyshq.net/yosys/
[45] Sadegh Yazdanshenas and Vaughn Betz. 2019. COFFE2: Automatic modelling and optimization of complex and heterogeneous FPGA architectures. *ACM Transactions on Reconfigurable Technology and Systems* 12, 1, Article 3 (Jan. 2019), 1–27.
[46] Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, Hongyi Wang, Wenheng Ma, Hanbo Sun, Shiyao Li, Zixiao Huang, et al. 2024. FlightLLM: Efficient large language model inference with a complete mapping flow on FPGAs. In *Proceedings of the 32nd ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*.

Received 15 January 2026; revised 29 April 2026; accepted 15 June 2026